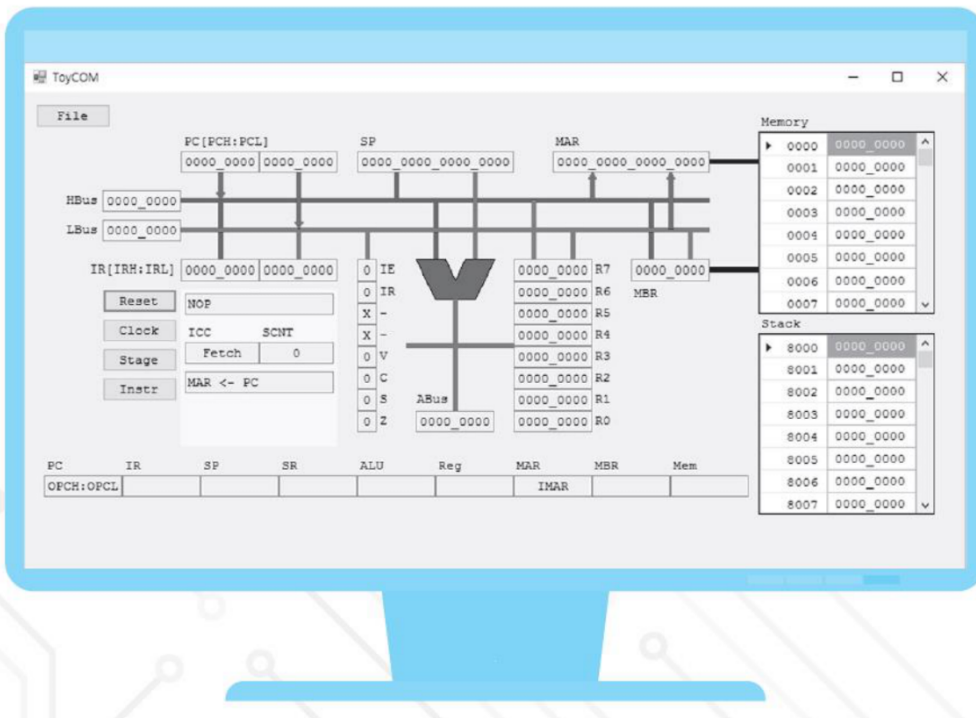


한눈에 보이는

개정 3 판

컴퓨터 구조



02. 논리회로 기초

2.1 수와 코드

2.2 조합논리회로

2.3 순차논리회로

2.4 레지스터 전송

2.5 요약

2.1 수와 코드

학습 내용

- 부호 없는 수(unsigned number)를 대상으로 컴퓨터가 수를 다루는 방법

내용

- 2.1.1 수의 체계
- 2.1.2 R 진수
- 2.1.3 진법 변환
- 2.1.4 2[✓]진수, 8~~진수~~, 16[✓]진수
- 2.1.5 코드
- 2.16 이진화 십진 코드
- 2.17 문자 코드

2.1.1 수의 체계

부호 없는 수(unsigned number)의 체계

수의 표현	...	1	2	3	4	.	5	6	...
자릿수	...	3	2	1	0		-1	-2	...
무게	...	10^3	10^2	10^1	10^0		10^{-1}	10^{-2}	...

$$\begin{aligned}(1234.56)_{10} &= 1 \times 10^3 + 2 \times 10^2 + 3 \times 10^1 + 4 \times 10^0 + 5 \times 10^{-1} + 6 \times 10^{-2} \\ &= 1000 + 200 + 30 + 4 + 0.5 + 0.06 \\ &= 1234.56\end{aligned}$$

2.1.2 R 진수

R 진법의 수 N의 표현:

$$N : (A_{n-1}A_{n-2}\cdots A_1A_0 \cdot A_{-1}A_{-2}\cdots A_{-m})_R$$

N의 크기

$$\begin{aligned} V(N) &= A_{n-1} \times R^{n-1} + A_{n-2} \times R^{n-2} + \cdots + A_{-m} \times R^{-m} \\ &= \sum_{k=-m}^{n-1} A_k \times R^k \end{aligned}$$

예

$$\begin{aligned} (1234.2)_5 &= 1 \times 5^3 + 2 \times 5^2 + 3 \times 5^1 + 4 \times 5^0 + 2 \times 5^{-1} \\ &= 1 \times 125 + 2 \times 25 + 3 \times 5 + 4 \times 1 + 2 \times 0.2 \\ &= 194.4 \end{aligned}$$

$$\begin{aligned} (427.3)_8 &= 4 \times 8^2 + 2 \times 8^1 + 7 \times 8^0 + 3 \times 8^{-1} \\ &= 4 \times 64 + 2 \times 8 + 7 + 3 \times 0.125 \\ &= 279.375 \end{aligned}$$

$$\begin{aligned} (1011.1)_2 &= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} \\ &= 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 + 1 \times 0.5 \\ &= 11.5 \end{aligned}$$

2.1.3 진법 변환

R 진수 $\rightarrow$ 10진수

- 수의 크기 계산 (2.1.2 R진수)

10진수 $\rightarrow$ R진수

- 진법이 변해도 크기는 변하지 않는다.

$$\begin{aligned} V(N_i) &= A_{n-1} \times 10^{n-1} + A_{n-2} \times 10^{n-2} + \cdots A_0 \times 10^0 \\ &= X_{k-1} \times R^{n-1} + X_{k-2} \times R^{n-2} + \cdots X_0 \times R^0 \end{aligned}$$

- 10 진수를 R로 나누어 R 진수를 구한다.

$$V(N_i) / R = X_{k-1} \times R^{n-2} + X_{k-2} \times R^{n-3} + \cdots X_1 \times R^0 \quad \text{remains} \quad X_0$$

진법 변환의 예

$$(527)_{10} \rightarrow 8\text{진수}$$

$$527 / 8 = 65 \dots 7 \quad (X_0 = 7)$$

$$65 / 8 = 8 \dots 1 \quad (X_1 = 1)$$

$$8 / 8 = 1 \dots 0 \quad (X_2 = 0)$$

$$1 / 8 = 0 \dots 1 \quad (X_3 = 1)$$

$$(527)_{10} = (1017)_8$$

$$(27)_{10} \rightarrow 2\text{진수}$$

$$27 / 2 = 13 \dots 1 \quad (X_0 = 1)$$

$$13 / 2 = 6 \dots 1 \quad (X_1 = 1)$$

$$6 / 2 = 3 \dots 0 \quad (X_2 = 0)$$

$$3 / 2 = 1 \dots 1 \quad (X_3 = 1)$$

$$1 / 2 = 0 \dots 1 \quad (X_4 = 1)$$

$$(27)_{10} = (11011)_2$$

2.1.4 2진수, 8진수, 16진수

2진수

- 사람이 다루기 불편
- $2^3 = 8$: 2 진수를 3 자리씩 읽으면 8 진수
- $2^4 = 16$: 2 진수를 4 자리씩 읽으면 16 진수

예

10110001101011.111100100

010_110_001_101_011.111_100_100

2 6 1 5 3. 7 4 4

0010_1100_0110_1011.1111_0010

2 C 6 B. F 2

2진수, 8진수, 16진수의 대응 관계

→ 16 = 0x10

10 진수	2 진수	8 진수	16 진수
0	0000	00	0
1	0001	01	1
2	0010	02	2
3	0011	03	3
4	0100	04	4
5	0101	05	5
6	0110	06	6
7	0111	07	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

2.1.5 코드(Code)

코드

- 유한 개의 원소로 구성된 집합의 원소들에게
- 서로 구분할 수 있는 유일한 수를 부여한 숫자

숫자 코드

- 10진수에 코드를 부여 한 것
- 이진화십진코드 (BCD, Binary Coded Decimal)

문자 코드

- 문자를 숫자로 표현한 것
- ASCII code, Unicode

2.1.6 이진화 십진 코드 BCD

10진수 기호(0~9)를 2진수로 표현

모두 10 개이므로 최소 4 비트 필요($2^3 < 10 < 2^4$)

1234₁₀

↓

0001 0010 0011 0100_{BCD}

0100 1101 0010₂

10 진수 기호	0	1	2	3	4	5	6	7	8	9
8421 BCD 코드	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
3 초과 BCD 코드	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100

X

예: 1225

- 8421 BCD 코드: 0001_0010_0010_0101
- 3 초과 코드: 0100_0101_0101_1000

유니코드: 16비트 체계 (2^{16} 개)

C/C++: 0x47

일반적: 47h

2.1.7 문자 코드

→ 알파벳 이외 연어용

표 2-3 아스키코드 표

→ 100 0111
G: 47_h → Hexa

b ₃ b ₂ b ₁ b ₀	b ₆ b ₅ b ₄							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

2.2 조합논리회로

학습 목표

- 조합논리회로의 기본 소자인 논리게이트의 동작을 이해한다.
- 논리회로의 동작을 해석할 수 있다.
- 조합논리회로 빌딩 블록의 동작을 이해한다.

내용

- 2.2.1 논리게이트
- 2.2.2 정논리와 부논리
- 2.2.3 조합논리회로 표현
- 2.2.4 디코더
- 2.2.5 인코더
- 2.2.6 멀티플렉서와 디멀티플렉서
- 2.2.7 가산기

2.2.1 논리게이트

논리게이트: 이진신호가 통과하는 문

표 2-4 기본 논리게이트의 동작특성표

입력		AND	NAND	OR	NOR	XOR	XNOR	NOT	BUFFER
X	Y	$F=X \cdot Y$	$F=(X \cdot Y)'$	$F=X+Y$	$F=(X+Y)'$	$F=X \oplus Y$	$F=(X \oplus Y)'$	$F=X'$	$F=X$
0	0	0	1	0	1	0	1	1	0
0	1	0	1	1	0	1	0		
1	0	0	1	1	0	1	0	0	1
1	1	1	0	1	0	0	1		

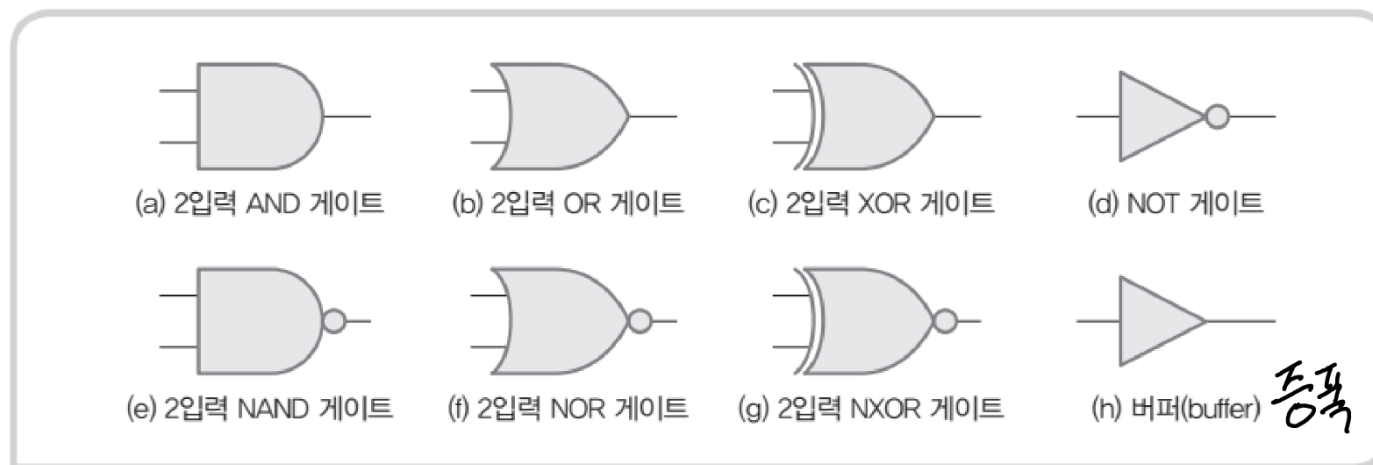


그림 2-2 논리게이트의 그래픽 기호

2.2.2 정논리와 부논리

positive logic Negative logic

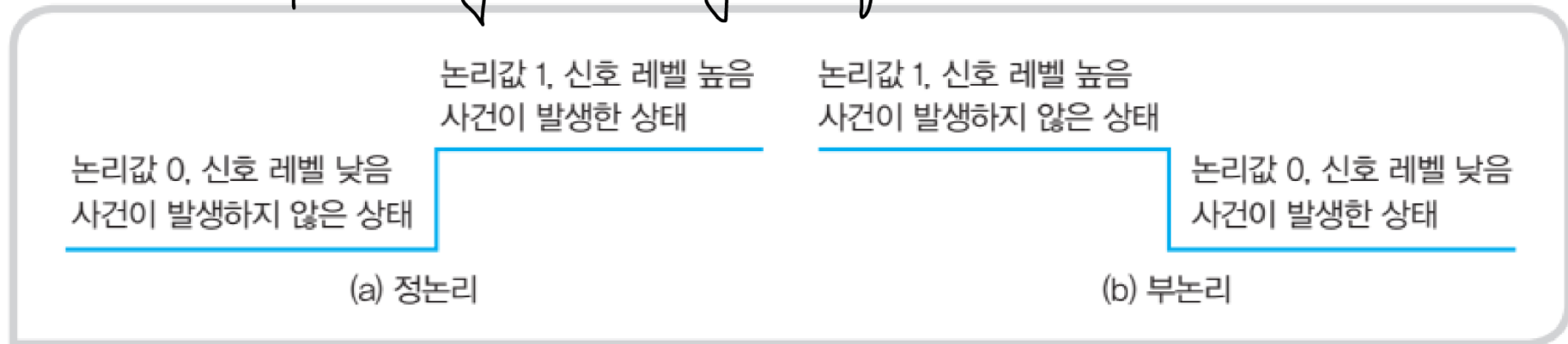


그림 2-3 이진신호의 정논리와 부논리

정논리 (positive logic)

- 신호 레벨이 high일 때 사건 발생
- 표현 방법: *alarm*.

부논리 (negative logic)

- 신호 레벨이 low일 때 사건 발생
- 표현 방법: $\overline{alarm}$, $/alarm$, 또는 $-alarm$

회로도에선 Bubble로 표현



2.2.3 조합논리회로 표현

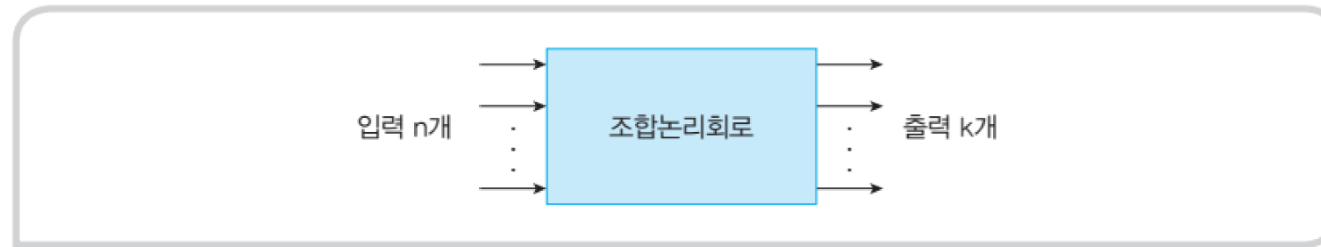


그림 2-4 조합논리회로의 입력과 출력

조합논리회로

- 입력 신호의 조합에 의하여 출력이 결정된다. *Combinational Circuit*
- 궤환 경로(feedback path)가 없다.

조합 논리회로 표현 방법

- 진리표 (truth table)
- 논리식 (logic equation)
- 논리 회로도 (logic diagram)

다수 함수(majority function)

다수 함수

- 세 개의 입력 신호: X, Y, Z
- 출력 F:
 - 값이 1인 신호가 값이 0인 신호보다 많으면, 1.
 - 아니면, 0.

입력			출력
X	Y	Z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

$$F(X,Y,Z) = X \cdot Y + Y \cdot Z + X \cdot Z$$

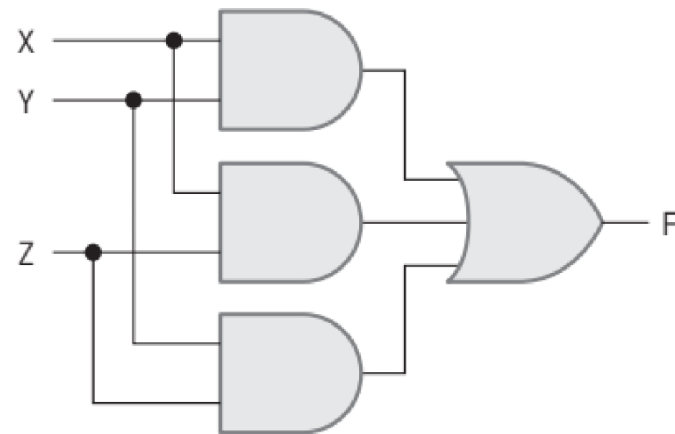


그림 2-5 다수 함수의 논리식, 논리회로도, 진가표

2.2.4 디코더 (Decoder)

코드를 푸는 장치 *작은 Input → 여러 개의 Output*

n비트의 이진 정보를 최대 2^n 개의 출력으로 변환하는 장치

$n \times m$ 디코더 ($m \leq 2^n$)

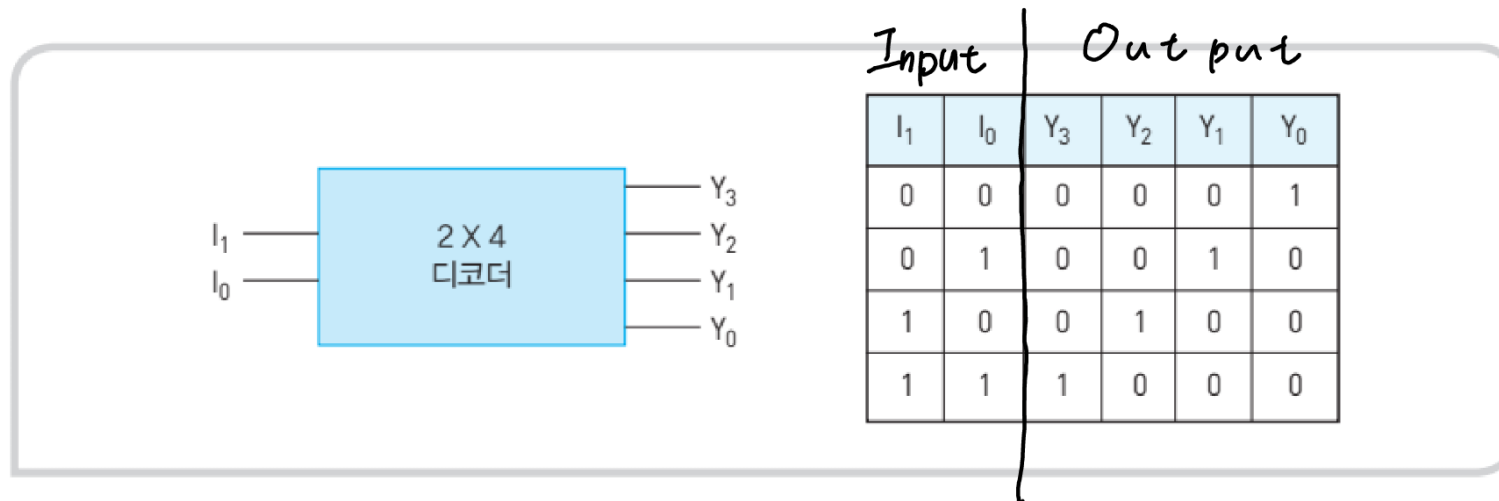


그림 2-6 정논리 2x4 디코더의 블록도와 동작특성표

인에이블 입력이 있는 디코더

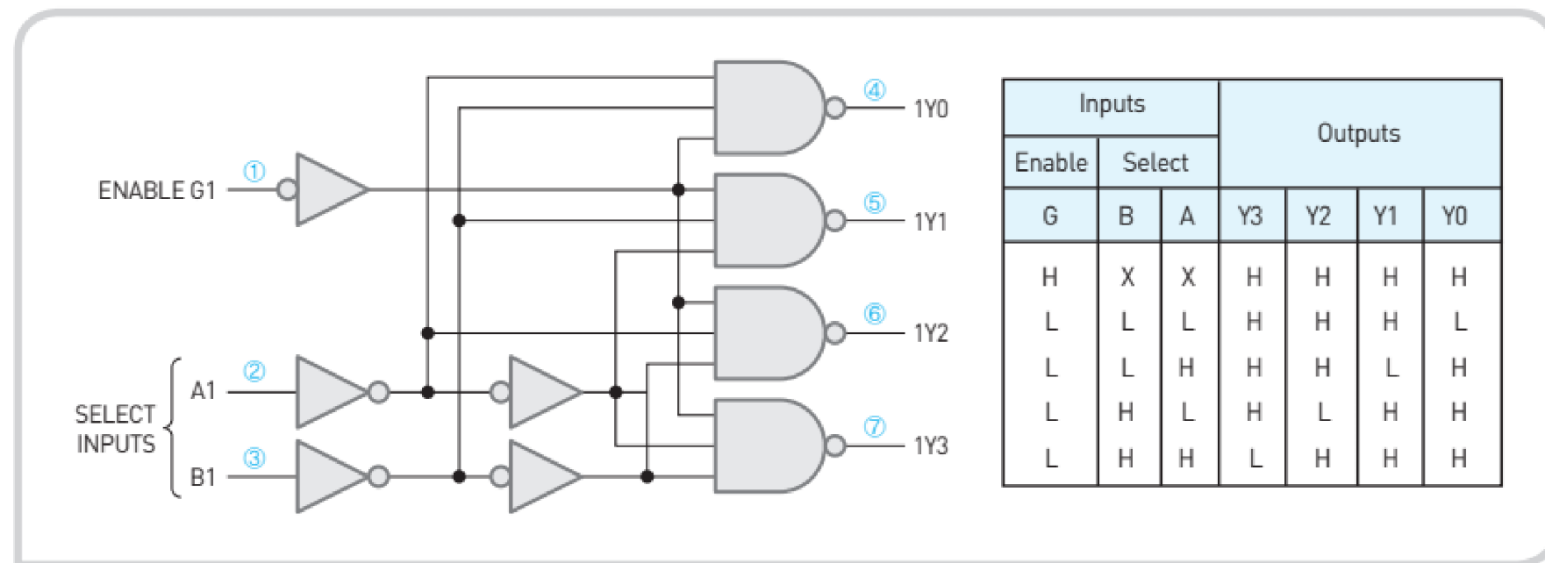


그림 2-7 인에이블 입력이 있는 2×4 디코더(74139)

2.2.5 인코더 (Encoder)

m 개의 입력에 대하여 n 비트의 코드를 생성하는 장치

8 x 3 인코더

- 입력 신호가 두 개 이상 동시에 1일 수 없다.

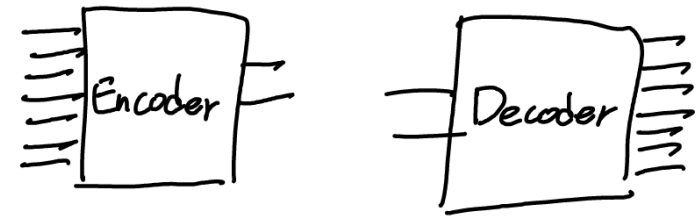


표 2-5 8x3 인코더

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

0번 핀 커짐: 출력 0

3번 핀 커짐: 출력 3

우선순위 인코더(Priority Encoder)

표 2-6 9 x 4 우선순위 인코더(74147)

[illegible]

2.2.6 멀티플렉서와 디멀티플렉서

MUX

DeMUX

멀티플렉서(multiplexer)

- 여러 개의 입력선 중에서 하나를 선택하여 출력으로 전달

디멀티플렉서(demultiplexer)

- 하나의 입력선을 여러 개의 출력으로 분배

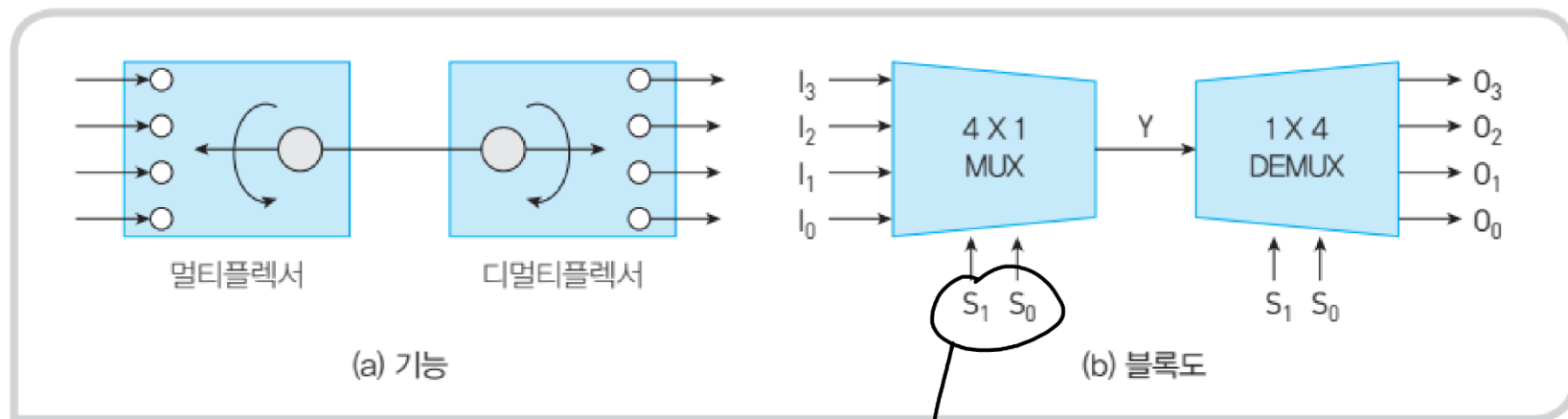


그림 2-8 멀티플렉서와 디멀티플렉서

→ 4개의 입력신호 제어용

동작 특성표

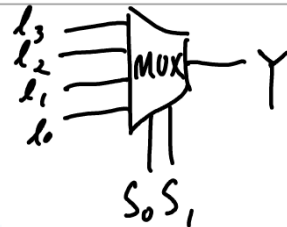


표 2-7 멀티플렉서와 디멀티플렉서의 동작특성표

멀티플렉서		
s_1	s_0	Y
0	0	l_0
0	1	l_1
1	0	l_2
1	1	l_3

디멀티플렉서					
s_1	s_0	o_3	o_2	o_1	o_0
0	0	0	0	0	Y
0	1	0	0	Y	0
1	0	0	Y	0	0
1	1	Y	0	0	0

2.2.7 가산기

반가산기 (half adder)

- 두 개의 한 비트를 더하여 합과 자리올림수를 생성하는 회로

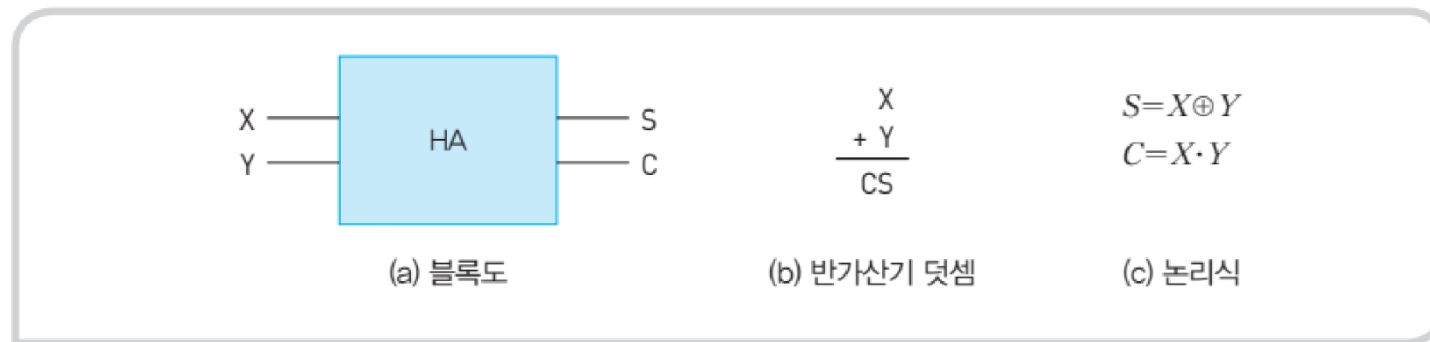


그림 2-9 반가산기

표 2-8 반가산기 동작특성표

입력		출력		설명
X	Y	C	Sum	
0	0	0	0	$0 + 0 = 00$
0	1	0	1	$0 + 1 = 01$
1	0	0	1	$1 + 0 = 01$
1	1	1	0	$1 + 1 = 10$

전가산기

전가산기 (full adder)

- 이전단에서 전달된 자리올림수까지 같이 더하는 회로

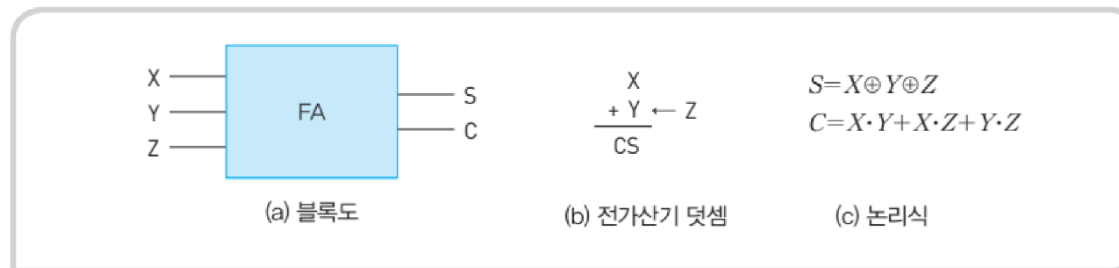


그림 2-10 전가산기

표 2-9 전가산기의 동작특성표

입력			출력		설명
X	Y	Z	C	S	
0	0	0	0	0	$0 + 0 + 0 = 00$
0	0	1	0	1	$0 + 0 + 1 = 01$
0	1	0	0	1	$0 + 1 + 0 = 01$
0	1	1	1	0	$0 + 1 + 1 = 10$
1	0	0	0	1	$1 + 0 + 0 = 01$
1	0	1	1	0	$1 + 0 + 1 = 10$
1	1	0	1	0	$1 + 1 + 0 = 10$
1	1	1	1	1	$1 + 1 + 1 = 11$

2.3 순차논리회로

학습 목표

- 플립플롭의 종류와 기능을 이해한다.
- 순차 논리회로 빌딩 블록의 동작을 설명할 수 있다.

구성

- 2.3.1 플립플롭
- 2.3.2 레지스터
- 2.3.3 시프트 레지스터
- 2.3.4 카운터

순차논리회로의 구조

순차 논리회로

- 기억 소자(flip-flop)를 포함한다.
- 출력 신호 = $F(\text{입력 신호}, \text{내부 상태})$

클럭 존재 → 회로 내 메모리 존재 → 순차 회로

동기 순차논리회로 (synchronous sequential circuit)

- 클럭 펄스로 제어하며, 기억 소자가 동시에 상태를 변경한다.

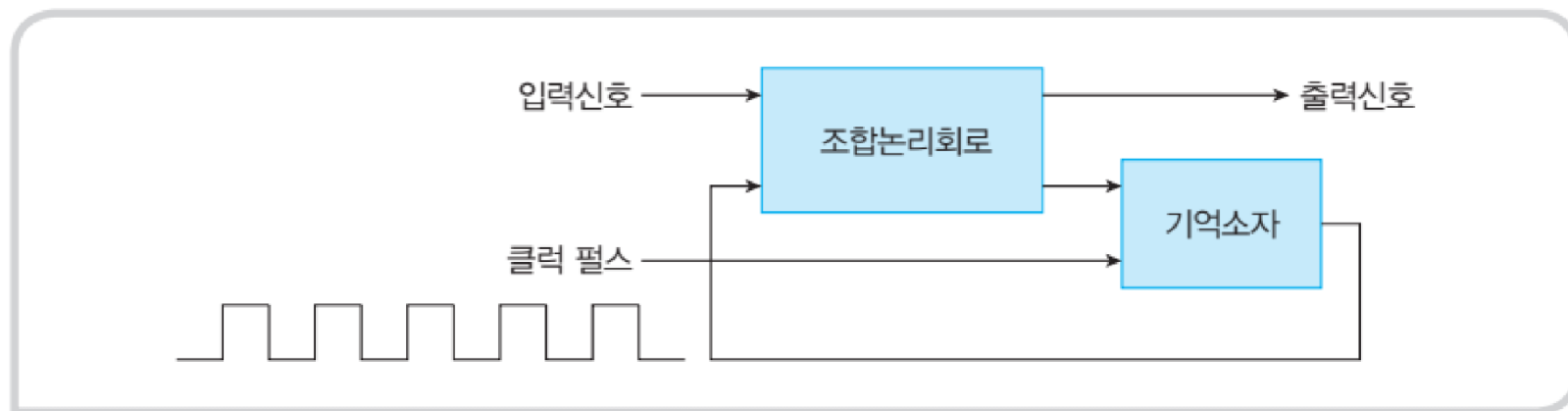


그림 2-11 동기 순차논리회로

2.3.1 플립플롭

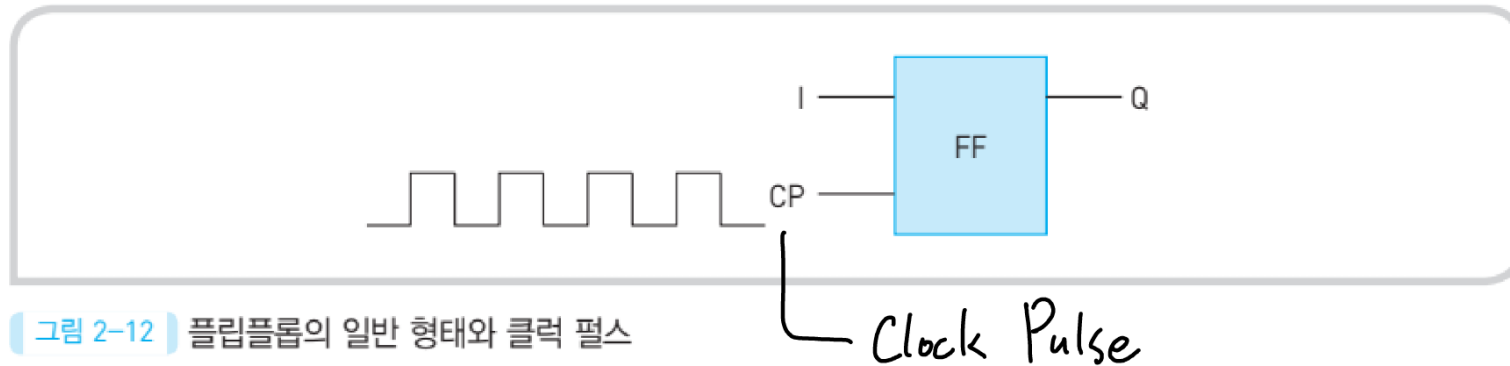
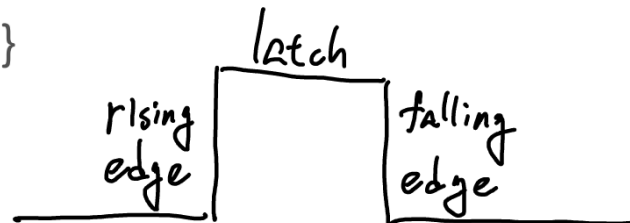


그림 2-12 플립플롭의 일반 형태와 클럭 펄스

플립플롭 (Flip-Flop)

- 한 비트의 정보를 저장하는 기억 소자
- 동작 시기: {latch, rising edge, falling edge}
- 종류: {D-FF, SR-FF, JK-FF, T-FF}



플립플롭의 동작

동작 시기

- 이벤트 직전의 입력값에 따라 직후의 출력이 변한다.

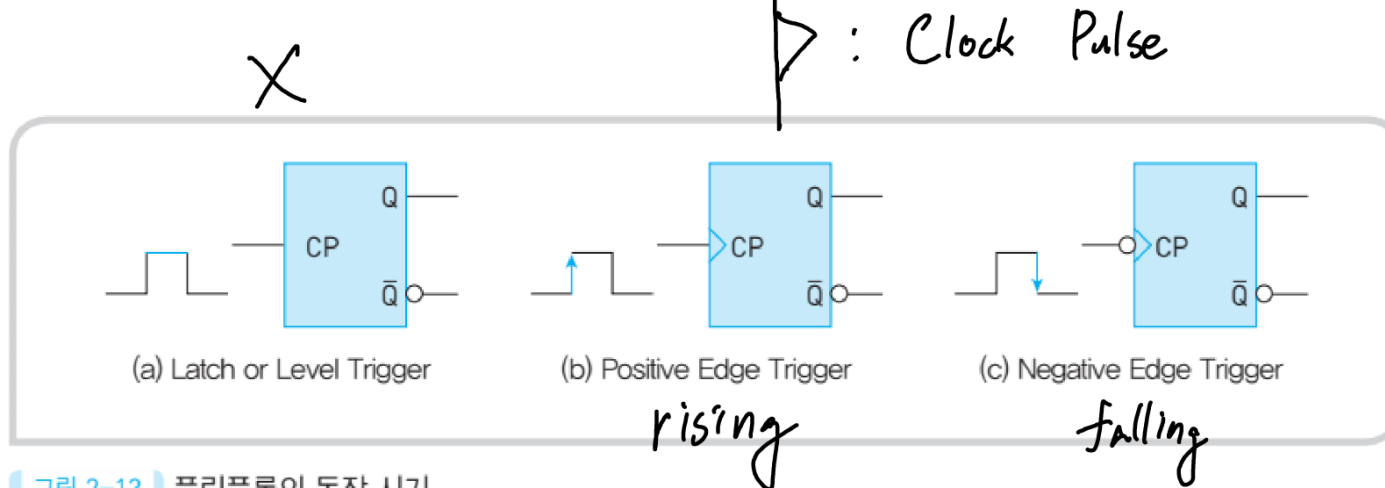


그림 2-13 플립플롭의 동작 시기

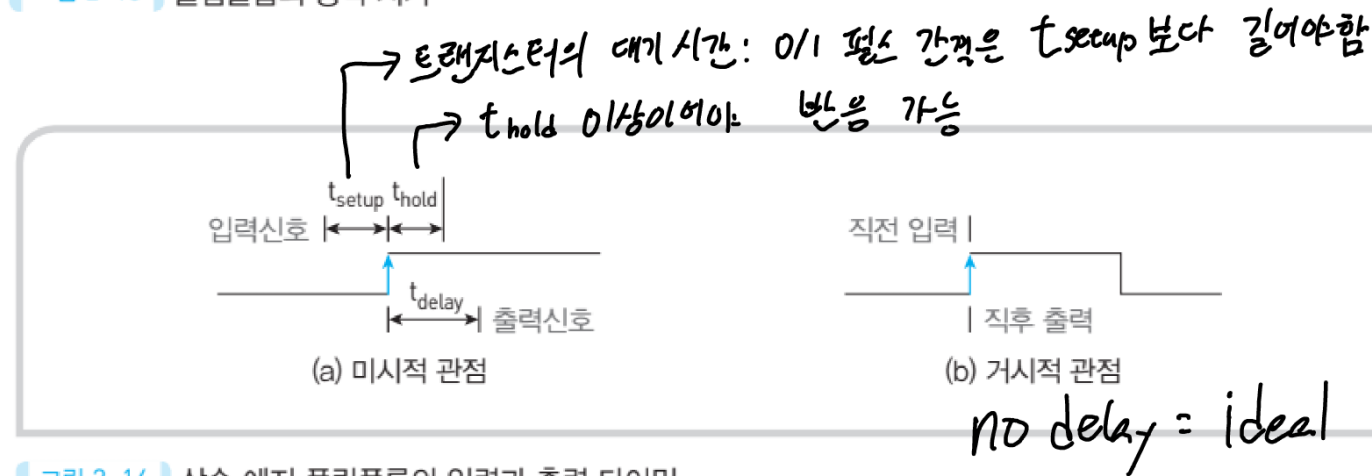
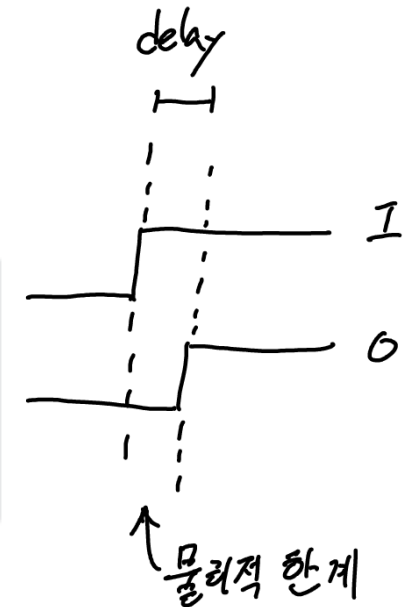


그림 2-14 상승 에지 플립플롭의 입력과 출력 타이밍



플립플롭의 종류

✓

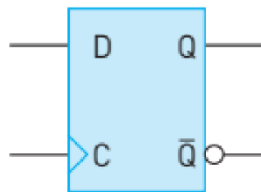
X

X

X

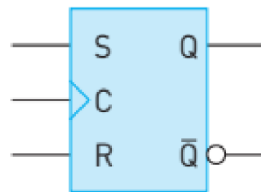
Set-Reset

Toggle



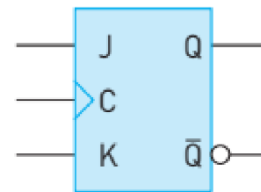
CP	D	Q
↑	0	0
↑	1	1

(a) D-FF



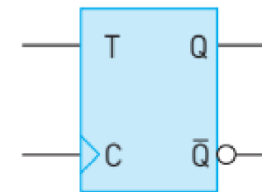
CP	S	R	Q
↑	0	0	Q
↑	0	1	0
↑	1	0	1
↑	1	1	NA

(b) SR-FF



CP	J	K	Q
↑	0	0	Q
↑	0	1	0
↑	1	0	1
↑	1	1	Q̄

(c) JK-FF



CP	T	Q
↑	0	Q
↑	1	Q̄

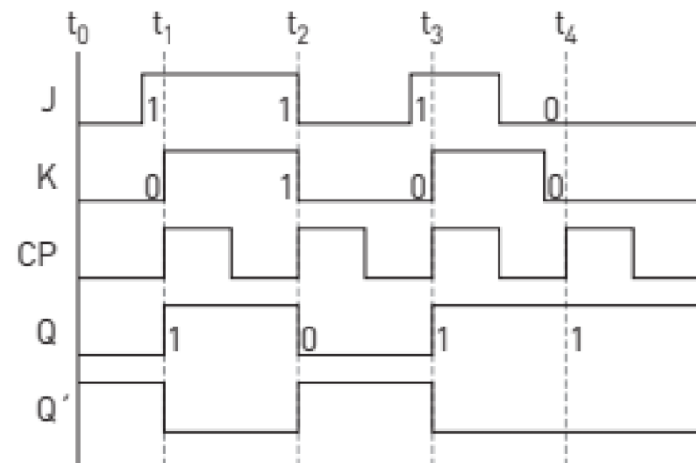
(d) T-FF

입력할 때만
플립플롭 값 변경

XOR과 동일

그림 2-15 플립플롭의 종류

JK-FF의 동작



CP	J	K	Q
↑	0	0	Q
↑	0	1	0
↑	1	0	1
↑	1	1	Q'

상승 에지 JK-FF 동작 특성표

그림 2-16 상승 에지 JK 플립플롭의 동작 예

2.3.2 레지스터(Register)

여러 비트의 이진 데이터를 저장하는 소자
클럭을 공유하여 동시에 동작

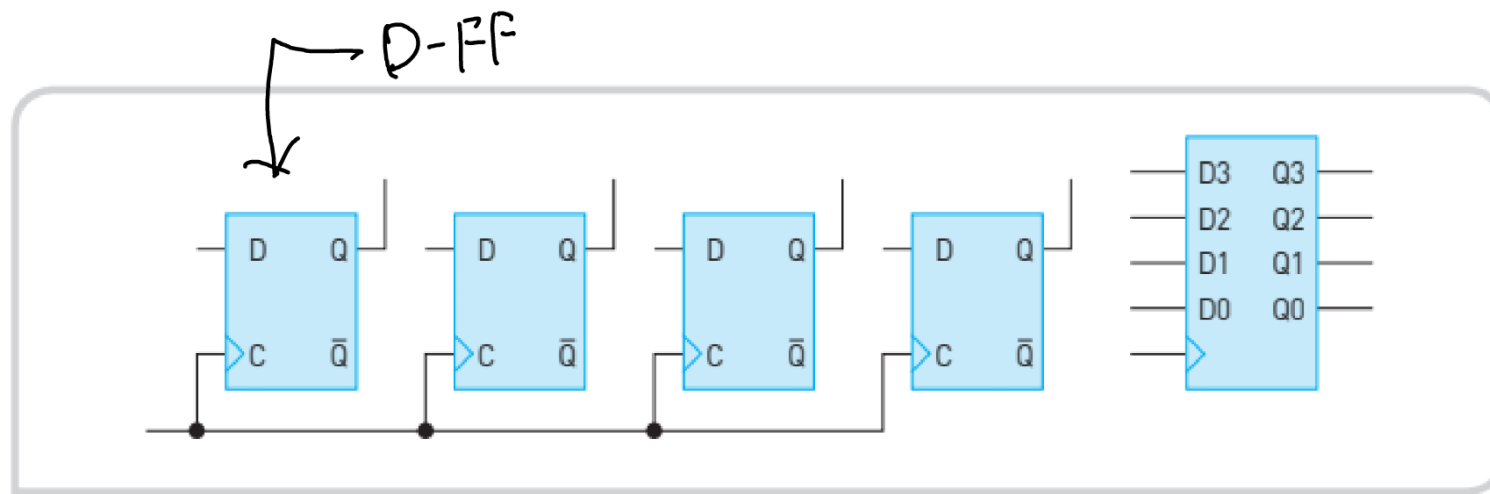


그림 2-17 4비트 레지스터

4비트 레지스터

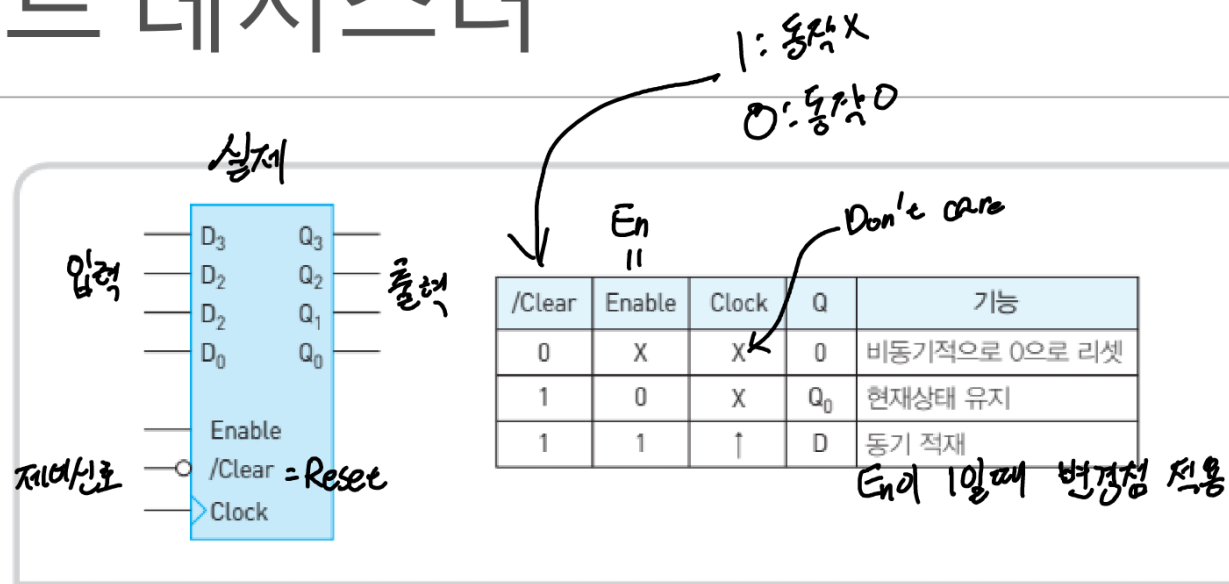


그림 2-18 비동기 리셋과 동기 적재 기능이 있는 4비트 레지스터

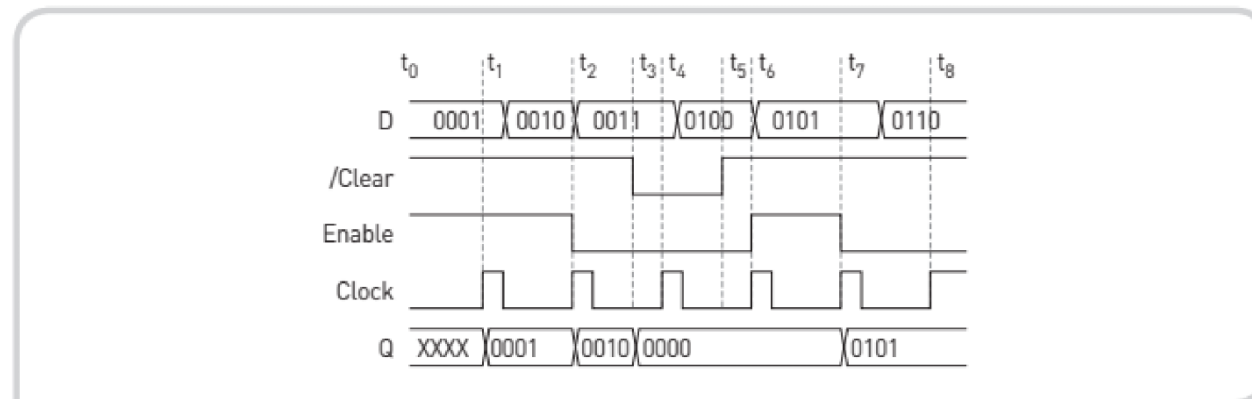


그림 2-19 4비트 레지스터의 동작

2.3.3 시프트 레지스터

데이터를 왼쪽 또는 오른쪽으로 자리 이동하는 레지스터

직렬 → 병렬 할 때 사용

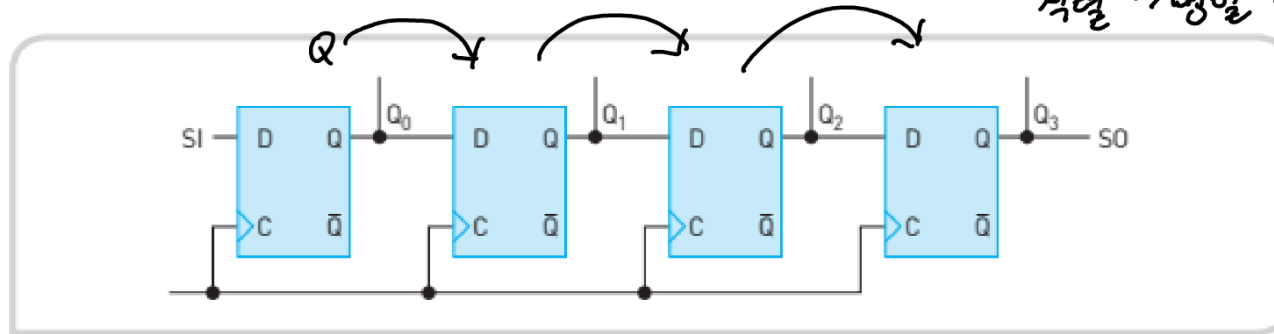


그림 2-20 간단한 시프트 레지스터

시프트 레지스터의 직렬 전송

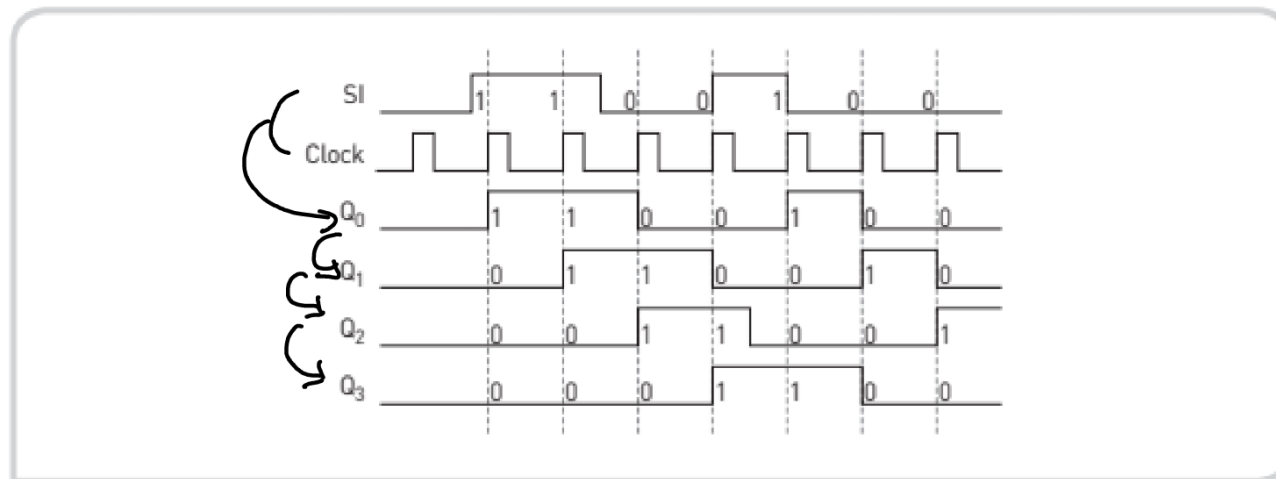


그림 2-21 시프트 레지스터의 동작

양방향 시프트 레지스터

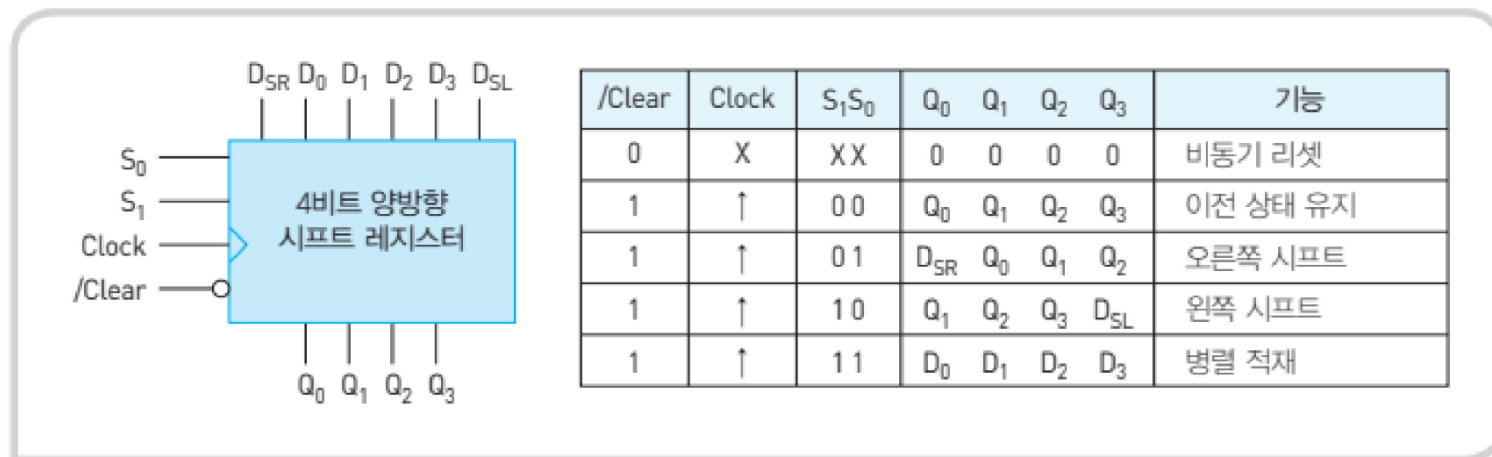
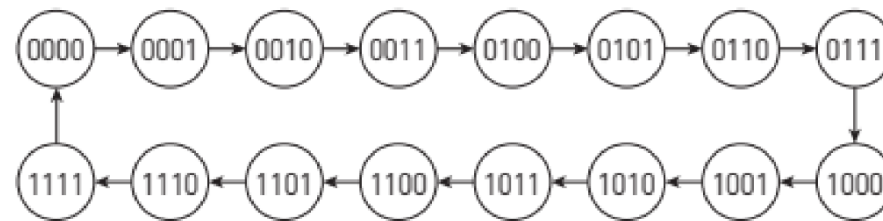


그림 2-22 4비트 양방향 시프트 레지스터

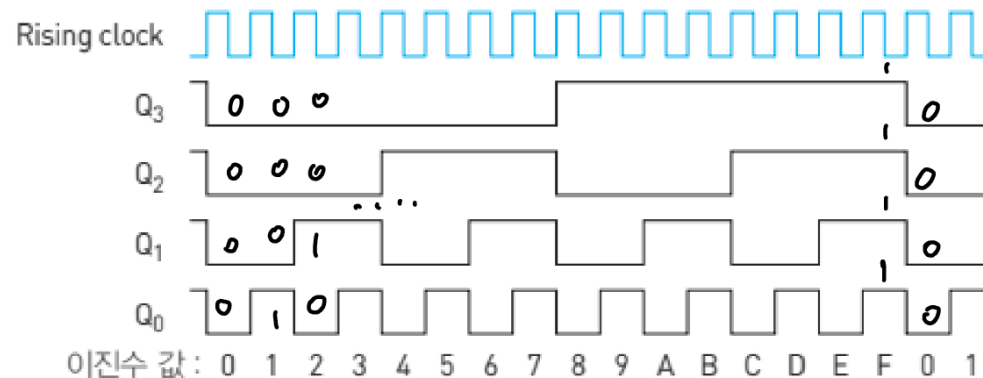
2.3.4 카운터(Counter)

미리 정해진 순서로 상태를 변경하는 레지스터

이진 카운터: 이진수 순서로 증가



(a) 상태도



(b) 출력 파형

4비트 카운터

병렬 적재 4 비트카운터

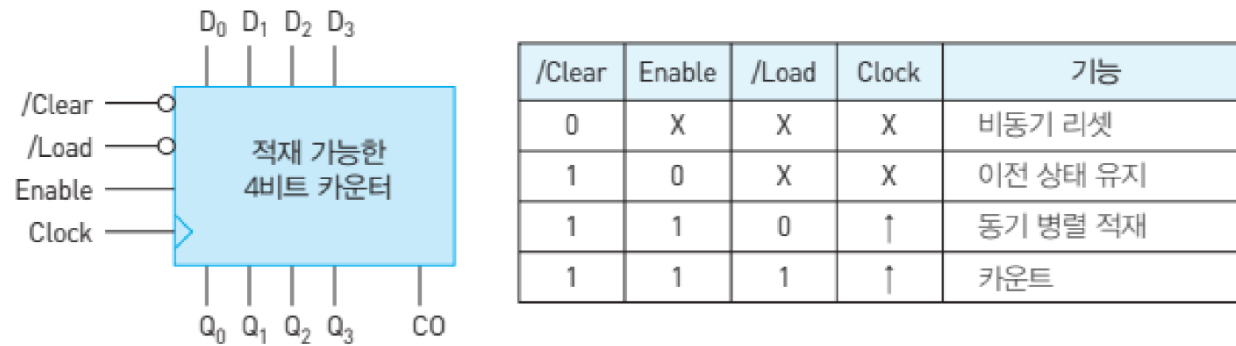


그림 2-24 병렬 적재 가능한 4비트 카운터

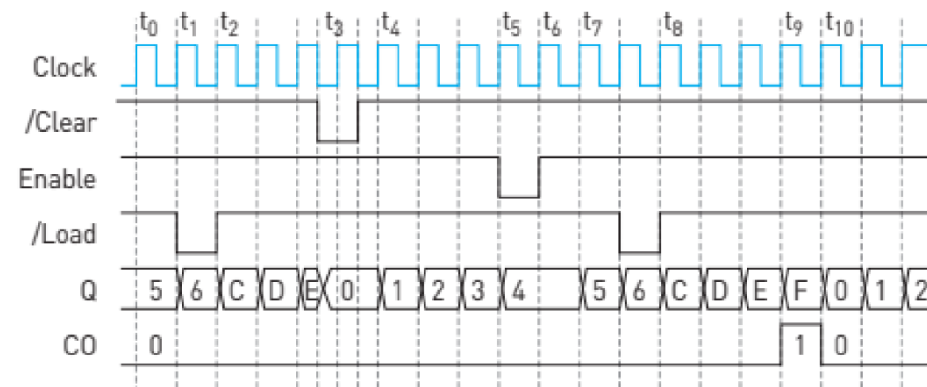


그림 2-25 병렬 적재 가능한 4비트 카운터의 동작 예

☆ 2.4 레지스터 전송 언어 RTL

학습 목표

레지스터 전송 언어

Register Transfer Language

- 하드웨어 회로가 마이크로오퍼레이션을 실행하는 방법을 이해한다.
CPU가 처리하는 최소언어
- 레지스터 간 데이터 전송 표현 방법을 익힌다.

내용

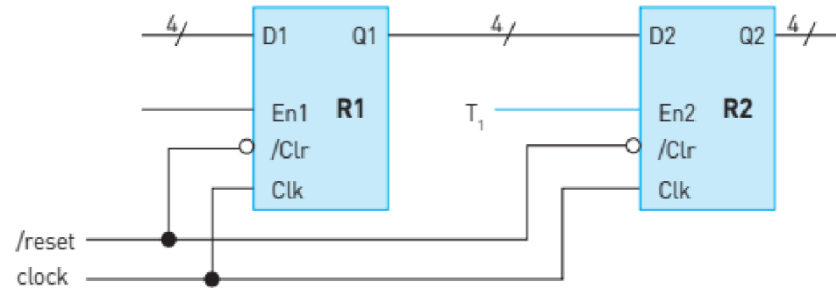
- 2.4.1 레지스터 간 데이터 전송
- 2.4.2 레지스터 전송 언어

2.4.1 레지스터 간 데이터 전송

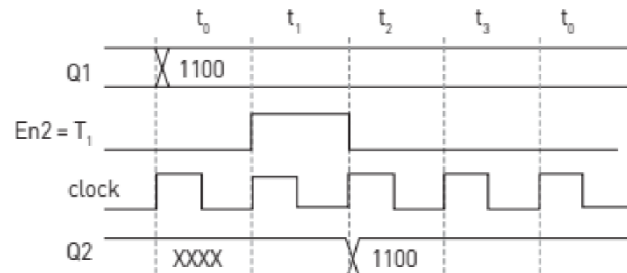
$T_1: R2 \leftarrow R1 \Rightarrow T_1$ 이 1일때 R_1 의 출력을 R_2 로 적재하는 동작의 표현

- $R1$ 의 출력을 $R2$ 의 입력으로 연결
- 제어장치의 타이밍 신호 T_1 을 $R2$ 의 적재 가능(enable)에 연결
- T_1 이후에 $R1$ 의 값을 $R2$ 로 전달

→ if ($T_1 == 1$) {
 $R_2 \leftarrow R_1$ }



(a) 레지스터 데이터 전달 회로

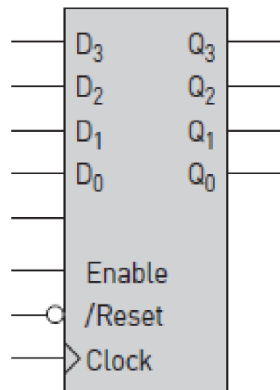


(b) 데이터 전송 타이밍 다이어그램

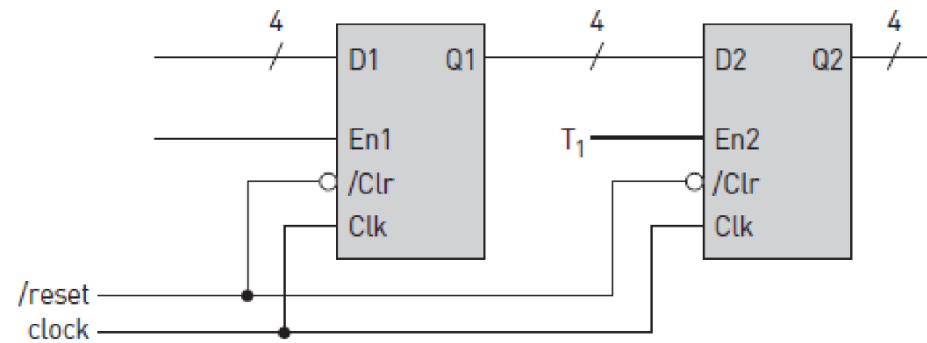
레지스터 간 데이터 전송

T1: $R2 \leftarrow R1$

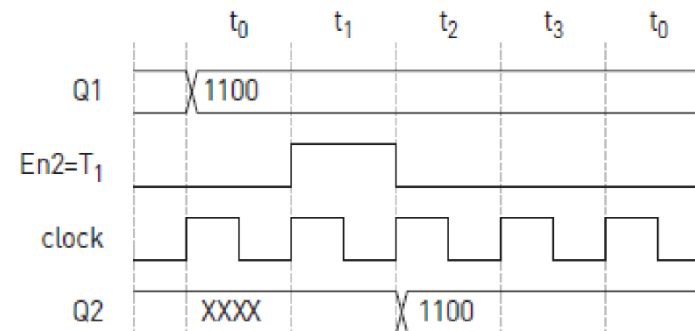
- R1의 출력을 R2의 입력으로 연결
- 제어장치의 타이밍 신호 T1을 R2의 적재 가능(enable)에 연결
- T1 이후에 R1의 값을 R2로 전달



< 4비트 레지스터 >



(a) 레지스터 데이터 전달 회로



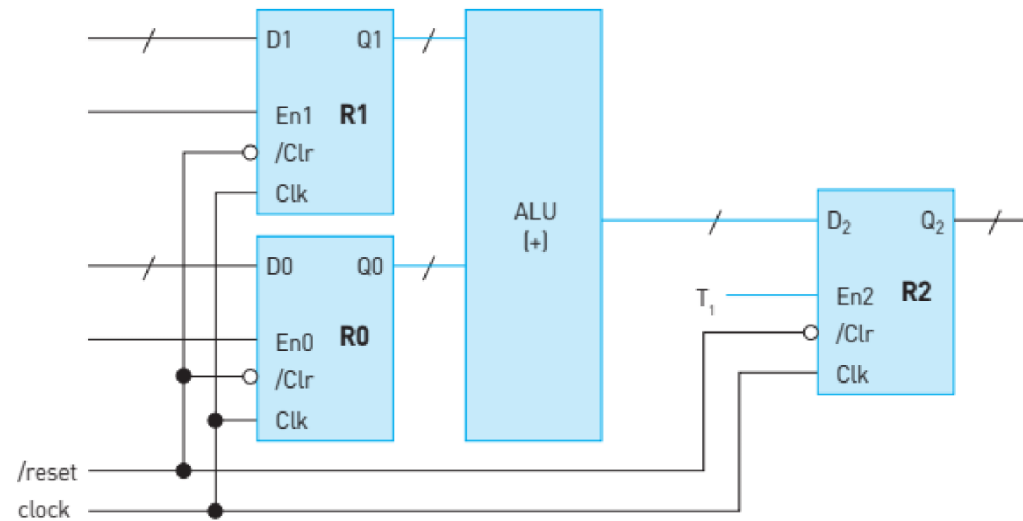
(b) 데이터 전송 타이밍 다이어그램

병렬 덧셈 $T_1: R2 \leftarrow R0 + R1$

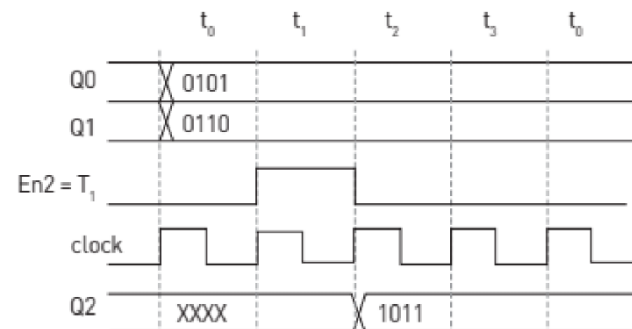
↙ T_1 이 일때 $R_0 + R_1$ 을 R_2 로 불러라

clock/reset

한 시스템의 모든 기억장치는
하나의 clock/reset을 공유
(대개 시스템에 1개밖에 없음)

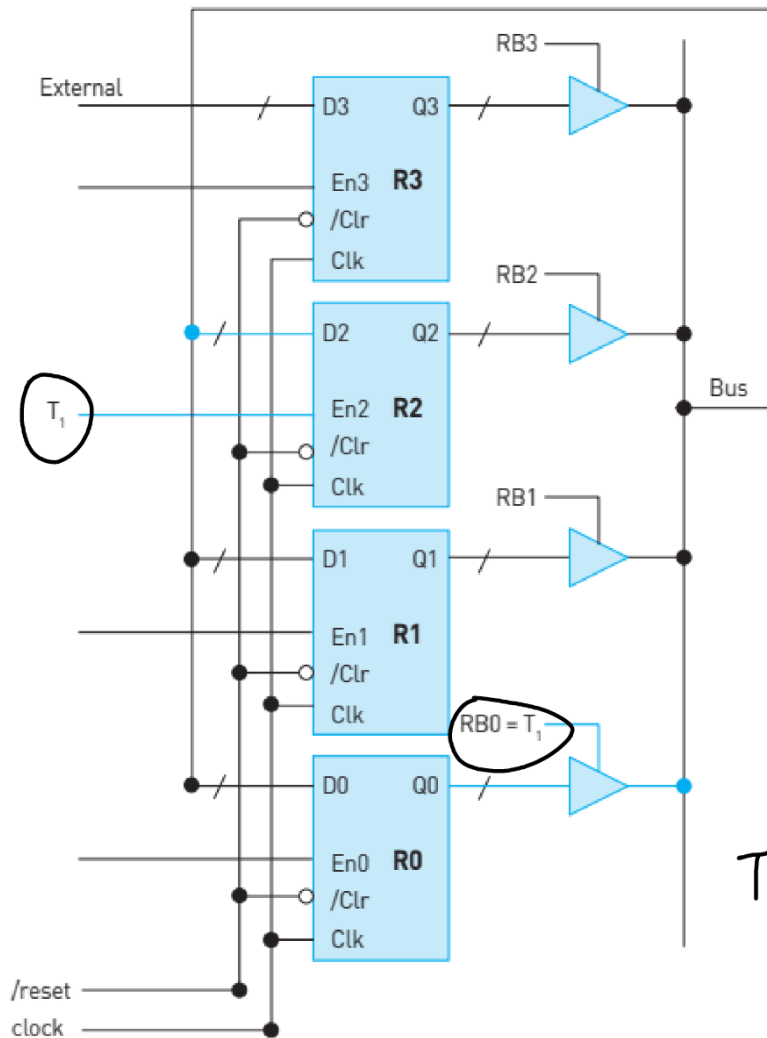


(a) 레지스터 덧셈 회로도

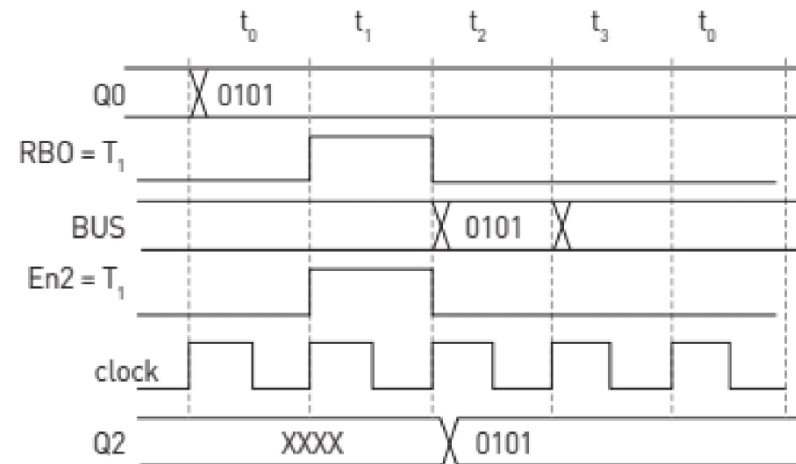


(b) 타이밍 다이어그램

버스 전송 $T_1: R2 \leftarrow R0$



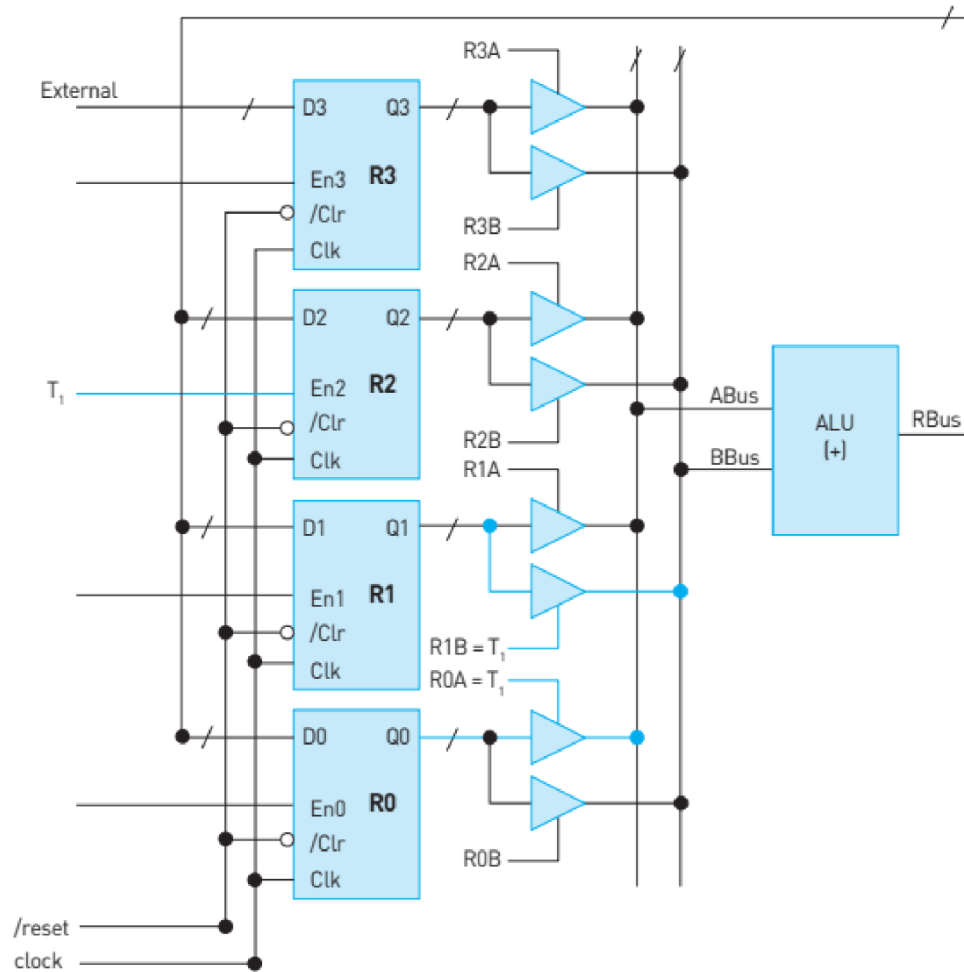
(a) 버스 연결 회로도



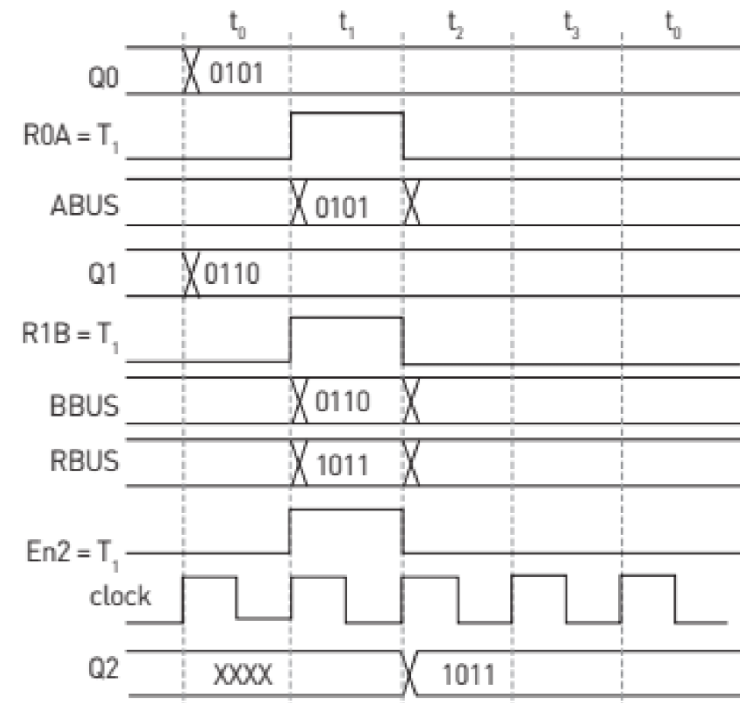
(b) 버스 타이밍 다이어그램

T_1 이 1일때 $R_2 \leftarrow Bus$ 에 넣어라

버스 연산 $T_1: R2 \leftarrow R0 + R1$



(a) 버스 연산 회로도



(b) 연산 타이밍 다이어그램

※ 화살표 방향은 '←' 이걸로 고정!

2.4.2 레지스터 전송 언어

레지스터 전송 언어(Register Transfer Language)

$T_2 + T_3: R_3 \leftarrow R_1 + R_2$ ※ 조건의 +: "더하기"

T_2 와 T_3 가 1일때 $R_1 + R_2$ 를 R_3 에 넣어라

if (START = 1), $R_1 \leftarrow 0$ ≡ START: $R_1 \leftarrow 0$

신호 START가 1이면 R_1 에 0을 넣어라

Symbol	Description	Examples
문자(숫자)	레지스터	MAR, PC, R2
괄호	레지스터의 일부	MBR(Opcode), R2(0-7)
화살표 ←	정보의 전달	$R2 \leftarrow R1$
кома (,)	마이크로 동작의 구분	$R2 \leftarrow R1, R1 \leftarrow R2$ R_1을 R_2에 넣는 동시에 R_2를 R_1에 넣어라
콜론 (:)	동작이 일어나는 조건 또는 시간	$t3: AC \leftarrow AC + MAR$ (같은 클럭에서) $t1+t2: R1 \leftarrow R2$

$IR \leftarrow MBR[\eta:0]$

t_3 가 1일때 $MAR + AC$ 를 AC 에 넣어라

⇒ MBR의 0~ η 비트를 IR에 넣어라

컴퓨터 설계 과정

데이터 처리부 구조 및 기능 정의



레지스터 전송 언어로 명령어 표현



하드웨어 및 제어장치 설계/구현